

HAI 课程项目开发任务：搭建 AI 虚拟聊天对象的基础 Pipeline

一、你的角色

你是一名资深 Python 工程师和 AI 应用原型开发者。

请帮助我搭建一个可运行、模块化、容易调试的 AI 虚拟聊天对象基础系统。该系统用于 Human-AI Interaction 课程项目。

当前阶段只负责基础 Pipeline，不负责复杂的个性化算法、长期记忆或正式用户实验。

项目暂定名称：

Emotion- and Gesture-Aware AI Chat Avatar

核心流程：

用户文本输入

- LLM 生成自然语言回复和控制标签
- 校验并标准化控制标签
- TTS 生成语音
- Avatar 播放语音
- Avatar 执行口型、表情和简单动作

请使用 Python 作为主要开发语言。

二、当前阶段目标

实现一个最小可运行版本 MVP，至少完成以下功能：

1. 用户在界面中输入文本。
2. LLM 返回自然语言回复。
3. LLM 同时返回结构化的情绪、表情、动作和语音风格标签。
4. 系统将回复文字转为语音。
5. 系统将语音文件发送给 Avatar 模块播放。
6. Avatar 模块根据标签切换表情。
7. Avatar 模块根据标签触发动作。
8. 支持基本日志、错误处理和降级策略。
9. 即使暂时没有可用的 Live2D 模型，也能通过 Mock Avatar 完成整条 Pipeline 测试。

当前优先级：

稳定运行 > 模块化 > 展示效果 > 高级功能

不要在第一阶段实现：

- 摄像头情绪识别
- 语音输入
- 用户打断
- 长期记忆
- RAG
- Big Five 个性化
- 多角色系统
- 自定义模型训练
- 复杂流式语音
- 自动动作生成模型
- 复杂 3D Avatar

三、技术原则

3.1 Python 主控

使用 Python 作为总控制层，负责：

- 用户输入
- LLM 请求
- 输出解析
- Action Planner
- TTS
- 音频文件管理
- Avatar 控制
- 日志
- 配置管理
- 错误恢复

Avatar 层可以通过以下任一方式实现：

- VTube Studio WebSocket API
- Live2D Web 页面
- 第三方 Avatar SDK
- 本地 Mock Avatar
- 其他经过验证可用的方案

不要把业务逻辑直接写死在 Avatar SDK 中。

3.2 模块必须可替换

请为以下模块定义统一接口：

- LLMProvider
- TTSPProvider

- AvatarController
- ActionPlanner
- ConversationService

后续应该可以方便替换：

- OpenAI API
- 本地 LLM
- Edge TTS
- OpenAI TTS
- CosyVoice
- Live2D
- VTube Studio
- Mock Avatar

不要让某个具体 Provider 的 SDK 类型传播到其他模块。

3.3 必须支持 Mock 模式

在没有 API Key、没有 Live2D 模型、没有 VTube Studio 的情况下，项目也必须能够运行。

Mock 模式要求：

- Mock LLM 返回固定或规则生成的结构化回复。
- Mock TTS 可以生成静音 wav、简单提示音，或跳过真实语音生成。
- Mock Avatar 在终端或网页显示：
 - 当前回复文本
 - 当前情绪
 - 当前表情
 - 当前动作
 - 当前语音风格
 - 音频文件路径
- Mock Avatar 应模拟动作开始、持续和结束。

Mock 模式用于验证完整 Pipeline，而不是仅验证单个模块。

四、推荐项目结构

请优先建立如下目录结构，可根据实现需要小幅调整，但必须保持清晰分层：

```
hai_chat_avatar/  
├── README.md  
├── pyproject.toml  
├── .env.example  
├── .gitignore  
├── config/  
│   └── default.yaml
```

```

|   └── action_mapping.yaml
|── assets/
|   ├── audio/
|   ├── avatar/
|   └── temp/
|── src/
|   ├── hai_avatar/
|   │   ├── __init__.py
|   │   ├── main.py
|   │   ├── app.py
|   │   ├── config.py
|   │   ├── schemas.py
|   │   ├── exceptions.py
|   │   ├── logging_config.py
|   │   ├── services/
|   │   │   ├── conversation_service.py
|   │   │   └── pipeline_service.py
|   │   ├── llm/
|   │   │   ├── base.py
|   │   │   ├── mock_provider.py
|   │   │   └── openai_provider.py
|   │   ├── tts/
|   │   │   ├── base.py
|   │   │   ├── mock_provider.py
|   │   │   └── edge_tts_provider.py
|   │   ├── avatar/
|   │   │   ├── base.py
|   │   │   ├── mock_controller.py
|   │   │   └── vtube_studio_controller.py
|   │   ├── planner/
|   │   │   ├── action_planner.py
|   │   │   ├── mapping.py
|   │   │   └── validator.py
|   │   └── ui/
|   │       └── gradio_app.py
|── tests/
|   ├── test_schemas.py
|   ├── test_action_planner.py
|   ├── test_pipeline.py
|   └── test_mock_providers.py
|── scripts/
|   ├── run_mock.py
|   ├── run_gradio.py
|   └── test_avatar_connection.py

```

五、核心数据结构

请使用 Pydantic 定义严格的数据结构。

5.1 LLM 原始结构化输出

建议定义：

```
class LLMAvatarResponse(BaseModel):
    reply_text: str
    emotion: str
    expression: str
    gestures: list[str]
    voice_style: str
    pause_before_speech_ms: int = 0
```

5.2 标准化后的控制指令

```
class AvatarCommand(BaseModel):
    emotion: EmotionType
    expression: ExpressionType
    gestures: list[GestureType]
    voice_style: VoiceStyleType
    gesture_intensity: float = 0.5
    speaking_rate: float = 1.0
    pause_before_speech_ms: int = 0
```

5.3 完整对话结果

```
class PipelineResult(BaseModel):
    user_text: str
    reply_text: str
    avatar_command: AvatarCommand
    audio_path: str | None
    latency_ms: dict[str, float]
    warnings: list[str]
```

六、第一版允许的标签集合

不要让 LLM 任意生成无限制标签。

第一版使用有限枚举。

情绪 Emotion

```
neutral
happy
```

supportive
thoughtful
confused
surprised
serious
apologetic

表情 Expression

neutral
smile
soft_smile
thinking
confused
surprised
concerned
serious

动作 Gesture

idle
nod
wave
head_tilt
think
explain
agree
small_bow

语音风格 Voice Style

neutral
calm
cheerful
gentle
serious
apologetic

每次回复最多允许两个动作。

如果 LLM 返回未知标签，必须经过标准化和降级：

- 未知 emotion → neutral
- 未知 expression → neutral
- 未知 gesture → idle
- 未知 voice_style → neutral

- 动作数量超过两个 → 只保留前两个
- 空回复文本 → 返回友好的错误提示或重试一次

七、LLM 输出格式

要求 LLM 只返回 JSON，不要返回 Markdown 代码块，不要附加解释。

目标格式：

```
{
  "reply_text": "没关系，我们可以先把任务拆成几个小步骤，一步一步完成。",
  "emotion": "supportive",
  "expression": "soft_smile",
  "gestures": ["nod", "explain"],
  "voice_style": "calm",
  "pause_before_speech_ms": 200
}
```

请设计一个稳定的 system prompt，并保存到独立文件或配置项中。

Prompt 中必须明确：

1. 回复应简洁自然。
2. 不得生成枚举以外的标签。
3. 最多两个动作。
4. 动作必须与回复语义匹配。
5. 安慰场景避免夸张动作。
6. 严肃场景避免持续微笑。
7. 疑惑场景可使用 confused 或 head_tilt。
8. 告别场景可使用 wave。
9. 解释场景可使用 nod 或 explain。
10. 不确定时选择 neutral 和 idle。

请对 JSON 解析失败实现：

- 第一次解析失败：尝试提取 JSON。
- 第二次失败：使用修复 Prompt 重试一次。
- 仍失败：降级到纯文本回复加 neutral 控制标签。
- 所有异常写入日志。

八、Action Planner

Action Planner 是后续项目的核心，但当前阶段先实现可靠的基础版。

请采用两层设计：

第一层：LLM 推荐标签

LLM 返回 emotion、expression、gestures 和 voice_style。

第二层：规则校验与纠正

规则层负责：

- 标签合法性检查
- 冲突检测
- 动作数量限制
- 不合理组合修复
- 默认动作补全
- 动作冷却时间
- 防止重复动作过多

第一版规则示例：

```
supportive + surprised -> 改为 supportive + soft_smile
serious + smile -> expression 改为 serious
apologetic -> 优先 small_bow
confused -> 可搭配 head_tilt
thoughtful -> 可搭配 think
happy -> 可搭配 smile 或 nod
告别类文本 -> 可搭配 wave
解释类文本 -> 可搭配 explain
```

不要完全依赖关键词规则生成全部动作。关键词规则只用于校验、纠错和降级。

为动作设置冷却机制，例如：

- wave: 10 秒内不重复
- small_bow: 10 秒内不重复
- think: 5 秒内不重复
- explain: 3 秒内不重复
- nod: 2 秒内不重复

如果动作仍在冷却中，替换为 idle 或 nod。

九、TTS 模块

第一版优先实现 Edge TTS Provider，并保留其他 Provider 接口。

接口示例：


```
class TTSProvider(ABC):
    @abstractmethod
    async def synthesize(
        self,
        text: str,
        voice_style: str,
        output_path: Path
    ) -> TTSResult:
        ...
```

TTSResult 至少包含：

```
class TTSResult(BaseModel):
    audio_path: str
    duration_ms: int | None
    sample_rate: int | None
```

第一版要求：

- 支持中文。
- 音频保存为 mp3 或 wav。
- 文件名使用 UUID 或时间戳，避免冲突。
- 语音生成失败时，界面仍展示文本。
- TTS 失败不能导致整个应用崩溃。
- 音频文件统一保存在 assets/temp 或运行时目录。
- 提供清理旧音频文件的函数。
- 不要把临时音频提交到 Git。

voice_style 第一版可通过语速、音高或音量近似映射：

```
neutral -> 默认
calm -> 稍慢
cheerful -> 稍快、稍高
gentle -> 稍慢、音量略低
serious -> 稍慢、音高略低
apologetic -> 稍慢、音量略低
```

如果 Edge TTS 无法稳定控制情感，不要声称实现了真实情感语音，只标记为“语音参数近似”。

十、Avatar 控制模块

定义统一接口：

```
class AvatarController(ABC):
    @abstractmethod
```

```

async def connect(self) -> None:
    ...

@abstractmethod
async def set_expression(self, expression: str) -> None:
    ...

@abstractmethod
async def trigger_gesture(self, gesture: str) -> None:
    ...

@abstractmethod
async def play_audio(self, audio_path: str) -> None:
    ...

@abstractmethod
async def start_speaking(self) -> None:
    ...

@abstractmethod
async def stop_speaking(self) -> None:
    ...

@abstractmethod
async def reset_to_idle(self) -> None:
    ...

```

10.1 Mock Avatar

必须优先完成 MockAvatarController。

其行为：

- 打印或在界面显示表情变化。
- 打印动作触发。
- 播放本地音频。
- 根据音频时长模拟说话状态。
- 输出类似日志：

```

[Avatar] Expression -> soft_smile
[Avatar] Gesture -> nod
[Avatar] Gesture -> explain
[Avatar] Speaking started
[Avatar] Playing audio: ...
[Avatar] Speaking stopped
[Avatar] Returned to idle

```

10.2 VTube Studio 或 Live2D

真实 Avatar 集成作为第二阶段。

在编写集成代码前，先验证：

- SDK 或仓库真实存在。
- 最新版本可以安装。
- 协议文档可访问。
- Python 是否能直接调用。
- 是否要求 Windows。
- 是否需要额外授权。
- 是否必须安装 VTube Studio。
- 是否需要用户手动确认插件权限。
- 是否支持触发表情热键。
- 是否支持参数控制。
- 是否能播放或获取音频振幅。

不要根据仓库名称猜测 API。

对于未经验证的 GitHub 项目，不要直接复制依赖或接口。先阅读 README、LICENSE、release 和最小示例。

真实 Avatar 第一版只要求：

- 连接成功
- 切换 3-5 个表情或热键
- 触发 3-5 个动作
- 播放 TTS 音频
- 实现基础口型同步或模拟口型

十一、口型同步策略

按实现难度依次尝试：

Level 1：模拟口型

播放音频时，以固定频率修改嘴巴开合参数。

适合先验证系统。

Level 2：音量驱动口型

读取音频振幅或实时播放音量，映射到 mouth_open 参数。

建议流程：

1. 读取音频。

2. 计算短时 RMS。
3. 归一化到 0-1。
4. 平滑处理。
5. 发送给 Avatar mouth-open 参数。

Level 3: 音素级口型

不作为当前阶段要求。

请先完成 Level 1 或 Level 2，不要在音素级同步上消耗过多时间。

口型同步必须与 AvatarController 解耦。

建议增加：

```
class LipSyncController:
    async def run(self, audio_path: str, avatar: AvatarController) -> None:
        ...
```

十二、Pipeline 执行顺序

推荐执行过程：

1. 接收 user_text
2. 校验输入
3. 调用 LLM
4. 解析结构化输出
5. Action Planner 标准化标签
6. 启动 TTS
7. Avatar 切换表情
8. Avatar 触发预说话动作
9. 等待 pause_before_speech_ms
10. 开始播放音频
11. 同时执行口型同步
12. 音频结束
13. 停止口型
14. Avatar 回到 idle
15. 返回 PipelineResult

第一版不要求 LLM、TTS 和 Avatar 完全并行。

优先保证执行顺序清楚且容易调试。

但请记录以下延迟：

- LLM latency
- JSON parsing latency
- Action Planner latency
- TTS latency
- Avatar preparation latency
- Audio playback duration
- End-to-end latency

十三、用户界面

第一版使用 Gradio。

界面至少包含：

- 用户文本输入框
- Send 按钮
- 对话历史
- AI 回复文本
- 当前 emotion
- 当前 expression
- 当前 gestures
- 当前 voice_style
- 生成的音频播放器
- Pipeline 状态
- 错误信息或警告
- Mock / Real Avatar 模式切换

不要先做复杂前端。

界面需要防止用户在一次请求尚未完成时连续提交多个请求。

可以采用：

- 禁用按钮
- 请求队列
- `asyncio.Lock`
- Gradio queue

十四、配置管理

使用 `.env` 和 YAML 配置。

`.env.example` 示例：

```
OPENAI_API_KEY=  
OPENAI_MODEL=  
LLM_PROVIDER=mock  
TTS_PROVIDER=mock  
AVATAR_PROVIDER=mock  
LOG_LEVEL=INFO
```

default.yaml 示例内容：

```
app:  
  language: zh-CN  
  max_input_chars: 500  
  max_reply_chars: 500  
  
llm:  
  provider: mock  
  temperature: 0.5  
  timeout_seconds: 30  
  max_retries: 1  
  
tts:  
  provider: mock  
  output_dir: assets/temp  
  voice: zh-CN-XiaoxiaoNeural  
  timeout_seconds: 30  
  
avatar:  
  provider: mock  
  reset_after_speech: true  
  default_expression: neutral  
  
planner:  
  max_gestures: 2  
  enable_cooldown: true
```

禁止：

- 在代码中写死 API Key
- 提交 `.env`
- 在日志中输出完整 API Key
- 在 README 示例中放真实密钥

十五、日志与错误处理

使用 Python logging。

日志至少包括：

- 请求开始
- 用户输入长度
- LLM Provider
- LLM 调用结果状态
- 原始结构化输出
- 解析结果
- 标签修复记录
- TTS 文件路径
- Avatar 连接状态
- 动作触发状态
- 各阶段耗时
- 异常堆栈

注意：

- 不要默认记录敏感的完整对话内容。
- 可以在 debug 模式记录完整文本。
- 普通模式只记录截断文本或哈希。
- API 请求失败时返回用户可理解的信息。
- 单个模块失败不应导致整个应用退出。

建议自定义异常：

```
LLMProviderError
LLMResponseParseError
TTSPProviderError
AvatarConnectionError
AvatarCommandError
PipelineError
```

十六、测试要求

请使用 pytest。

至少实现以下测试。

单元测试

1. 合法 LLM JSON 可以解析。
2. Markdown 包裹的 JSON 可以修复。
3. 未知 emotion 会降级为 neutral。
4. 未知 gesture 会降级为 idle。
5. 动作超过两个会被截断。
6. 冲突标签可以被纠正。
7. 动作冷却逻辑有效。

- 8. Mock TTS 返回有效文件路径。
- 9. Mock Avatar 能完整执行一次命令。

集成测试

输入：

我最近项目压力有点大，不知道怎么开始。

期望：

- 返回非空回复文本。
- emotion 属于允许集合。
- expression 属于允许集合。
- gestures 不超过两个。
- 生成音频或 Mock 音频。
- Avatar 完成一次表情和动作执行。
- PipelineResult 包含各阶段耗时。
- 即使真实 API 不可用，也能在 Mock 模式通过。

再增加以下场景：

你好！

谢谢你的帮助，再见。

我不太明白这个概念，你能解释一下吗？

对不起，我刚才可能说错了。

这真是个令人惊讶的结果。

十七、README 要求

README 必须包含：

1. 项目介绍
2. 系统架构图
3. Pipeline 流程
4. 目录结构
5. Python 版本
6. 安装方式

7. 环境变量配置
8. Mock 模式运行方式
9. Gradio 运行方式
10. 真实 LLM 配置方式
11. Edge TTS 配置方式
12. Avatar 集成状态
13. 已知限制
14. 测试运行方式
15. Demo 示例
16. 后续计划

运行方式应尽量简单，例如：

```
python -m venv .venv
source .venv/bin/activate
pip install -e .
python scripts/run_mock.py
```

Windows 命令也要给出。

十八、分阶段开发顺序

请严格按以下顺序推进，不要一开始同时实现所有 Provider。

Phase 0: 环境检查

- 检查 Python 版本。
- 检查操作系统。
- 检查是否有 GPU，但基础版本不依赖 GPU。
- 检查 ffmpeg、音频播放能力。
- 检查已有项目文件。
- 不删除已有文件。
- 不覆盖用户代码，除非明确说明。

Phase 1: 项目骨架

- 创建目录结构。
- 创建 pyproject.toml。
- 创建配置系统。
- 创建 Pydantic schemas。
- 创建基础接口。
- 创建日志模块。
- 创建异常类型。

验收标准：

```
python -c "import hai_avatar"
```

可以成功执行。

Phase 2: 完整 Mock Pipeline

实现：

- MockLLMProvider
- MockTTSPProvider
- MockAvatarController
- ActionPlanner
- PipelineService
- CLI Demo

验收标准：

```
python scripts/run_mock.py
```

输入一句中文后，可以完成整条流程。

Phase 3: Gradio 界面

- 文本输入
- 对话历史
- 控制标签显示
- 音频播放
- 状态显示
- 错误提示

验收标准：

```
python scripts/run_gradio.py
```

可以在浏览器中进行一轮完整交互。

Phase 4: 真实 LLM

- 实现 OpenAIProvider 或其他经过确认的 Provider。
- 强制结构化输出。
- 解析失败重试。
- 超时处理。
- 降级到 Mock 或 neutral。

验收标准：

真实 LLM 可以稳定返回符合 schema 的结果。

Phase 5: 真实 TTS

- 实现 EdgeTTSProvider。
- 保存音频。
- 显示音频。
- 测量耗时。
- 支持基础语速映射。

验收标准：

中文回复可以生成可播放音频。

Phase 6: 真实 Avatar

- 验证目标 Avatar 方案。
- 实现连接。
- 实现表情。
- 实现动作。
- 实现音频播放。
- 实现基础口型同步。

验收标准：

至少支持：

neutral
smile
thinking
confused
surprised

至少支持：

idle
nod
wave
head_tilt
think

Phase 7: 稳定性处理

- 加超时
- 加重试
- 加音频清理
- 加动作冷却
- 加重复提交保护
- 加集成测试
- 整理 README

十九、关于现有 GitHub 项目的要求

目前有若干候选项目或仓库，例如 Live2D、VTube Studio、Avatar SDK、AI VTuber、persona engine 等。

不要默认这些项目一定存在、兼容或适合直接 Fork。

在采用任何外部仓库前，请完成以下检查：

1. 仓库 URL 是否真实可访问。
2. README 是否包含可运行示例。
3. LICENSE 是否允许课程项目使用。
4. 最近 commit 和 release 是否正常。
5. Issues 是否显示项目已经不可用。
6. 所需运行平台。
7. 是否依赖闭源模型或付费软件。
8. 是否要求特殊 Live2D 模型。
9. 是否可以独立调用 emotion 和 gesture。
10. 是否可以由 Python 主控。
11. 是否与当前 Python 版本兼容。
12. 是否会引入过多无关功能。

如果候选仓库过于庞大，优先参考其架构，不要整仓 Fork。

如果必须 Fork，请先说明：

- 为什么 Fork
- 需要保留哪些模块
- 删除哪些无关模块
- 修改哪些文件
- 许可证要求
- 与本项目 Action Planner 的接口位置

不得因为某个仓库宣称支持 Emotion Engine，就把它直接视为本项目的创新部分。

二十、编码规范

- Python 3.11 或兼容版本。
- 使用类型注解。
- 使用 async/await 处理网络和 TTS 操作。
- 使用 pathlib，不要到处拼接字符串路径。
- 函数保持单一职责。
- 不要创建超大 main.py。
- 不要写大量全局变量。
- 不要静默吞掉异常。
- 不要在 import 阶段启动服务。

- 不要使用裸 `except:`
- 不要把 Provider 特定逻辑写进 PipelineService。
- 公共类和关键函数写 docstring。
- 对重要逻辑增加注释，但不要过度注释显而易见的代码。
- 所有外部依赖写入 pyproject.toml。
- 尽量避免不必要的大型依赖。

建议使用：

- pydantic
- pydantic-settings
- gradio
- httpx
- edge-tts
- pytest
- pytest-asyncio
- python-dotenv
- PyYAML

如需音频处理，可考虑：

- soundfile
- numpy
- pydub
- simpleaudio

但引入前先确认 Windows 环境安装是否稳定。

二十一、你每一步的输出格式

不要一次生成整个系统后才汇报。

每完成一个 Phase，请输出：

1. 本阶段目标
2. 新增或修改的文件
3. 核心设计决定
4. 如何运行
5. 测试结果
6. 当前已知问题
7. 下一阶段建议

修改文件前，先检查当前目录中的已有代码。

生成代码后，必须实际运行：

```
pytest
```

以及对应的启动命令。

不要只声称测试通过。请给出真实终端结果摘要。

如果某个依赖或外部服务不可用：

- 不要伪造运行结果。
- 说明具体失败原因。
- 保留 Mock 降级方案。
- 继续完成不依赖该服务的部分。

二十二、当前立即执行的任务

现在先完成 Phase 0、Phase 1 和 Phase 2。

具体要求：

1. 检查当前工作目录。
2. 检查已有文件和 Python 环境。
3. 创建模块化项目骨架。
4. 实现全部 Pydantic schema。
5. 实现 Provider 抽象接口。
6. 实现 MockLLMProvider。
7. 实现 MockTTSPProvider。
8. 实现 MockAvatarController。
9. 实现基础 ActionPlanner。
10. 实现 PipelineService。
11. 创建命令行交互 Demo。
12. 创建基础配置文件。
13. 创建 `.env.example`。
14. 创建单元测试和集成测试。
15. 实际运行测试。
16. 实际运行一次 Mock Pipeline。
17. 更新 README。

Mock LLM 不要只返回同一句话，可以根据简单场景生成不同结果：

- greeting
- supportive
- explanation
- confusion
- farewell
- apology
- surprise
- fallback

最后给出：

- 完整文件树
- 启动命令
- 测试摘要

- 当前尚未实现的功能
- 进入 Phase 3 前需要注意的问题